

|      |                     |
|------|---------------------|
| 任务号  | 5                   |
| 姓名   | 杨力泉 元杰              |
| 学号   | 231220069 231220082 |
| 日期   | 2026 年 4 月 2 日      |
| 附加任务 | 完成八进制与十六进制的识别       |

# 实验报告 — 语法分析器实现与调试

## 一、实验目标与概述

本实验目标是实现一个基于 Flex/Bison 的简化 C-like 语言（C--）的前端：包含词法分析、语法分析、错误报告以及按指定格式输出抽象语法树（AST）。要求严格控制错误信息与 AST 的文本格式，以满足自动化评测系统的精确比对。

本次实现主要修改与维护的文件位于 `Lab1.0/Code/`，核心文件包括：`lexical.l`（词法分析器）、`syntax.y`（语法/语义动作）、`main.c`（AST 实现与程序入口）。

## 二、总体设计与数据结构

### 1. AST 结点 (Node)

- 每个结点包含：名字 (name)、源代码行号 (line)、是否为终结符 (is\_terminal) 以及子结点列表或终结符值 (ID、INT、FLOAT、STRING)。
- 非终结符节点保存一个子结点数组；终结符节点保存具体值 (字符串或数值)。
- 提供 `create_node`、`create_terminal_node`、`create_id_node`、`create_int_node`、`create_float_node`、`create_string_node`、`print_tree`、`free_node` 等函数，用于语法动作中构建与打印树。

### 2. 错误控制

- 全局变量 `has_error`（由词法或语法报错时置 1）用于控制：若解析过程中任一错误发生，则最终不打印 AST（自动评测要求）。
- 词法错误（类型 A）与语法错误（类型 B）均输出到 `stdout`，格式严格按照评分脚本要求，例如：
  - `Error type A at Line %d: ...`（词法错误）
  - `Error type B at Line %d: ...`（语法错误或由 `bison` 报出的语法错误）

## 三、词法分析 (`lexical.l`) 实现

- 将不同数字格式（十进制、八进制、十六进制、小数）在词法阶段转换为对应的整数或浮点值，交由语法动作作用 `INT_CONST` / `FLOAT_CONST` 的终结符值。
- 对非法字面量（如非法八进制/十六进制）做即时报错（Error type A），并返回一个安全的 `INT_CONST(0)`，以便语法继续解析。
- 对“以数字开头的标识符”（如 `1bad`）进行特殊处理：报告 Error type A，然后尽可能提取后缀作为合法 ID 继续返回，或退化为整型常量，帮助解析器恢复。

## 四、语法分析 (syntax.y) 实现

- 使用 Bison 的 `%union` 和 `%type` 保证各非终结符有正确的语义值（均为 `Node*`）。
- 为解决 dangling-else 问题，采用 `matched/unmatched` 语法分支，并用 `%nonassoc LOWER_THAN_ELSE` 指定优先级。
- 语法动作严格构造 AST：终结符在必要时被包装成 `create_terminal_node("NAME", line)`；某些非终结符（例如 `Specifier`、`VarDec`、`DecList`、`Dec`、`OptTag`、`Tag` 等）根据评测需求被封装，以保证文本化树的节点顺序和标签完全匹配。
- 特殊错误模式：对缺少 `]`、缺少 `;` 等情况使用带 `error` 的产生式捕获并输出相应的 Error type B 信息，然后恢复解析（使用 `yyerrok`）。

示例：在 `Exp LB error RB` 的产生式中，会打印：

```
Error type B at Line %d: Missing "].
```

并设置 `has_error = 1`；以阻止最终 AST 打印。

## 五、遇到的主要困难与解决方案

### 1. Bison 的类型错误 ("`$<...>` has no declared type")

- 原因：未为很多非终结符声明 `%type`。解决：统一使用 `%union { struct Node* node; }` 并为所有产生式的左侧声明 `%type <node>`。

### 2. AST 文本对齐与标签严格匹配（评测要求精确到节点名与缩进）

- 困难：评测样例期望在某些语境中出现 `OptTag (line)`，在另一些语境中为 `Tag (line)` 或直接 `ID: name`。任何不精确的包装都会导致测试失败。
- 解决：逐个查看测试用例，最小化并精确调整语法动作：
  - 使 `OptTag` 在定义（带 `{ ... }`）的情形下以 `OptTag -> ID` 显示；
  - 使 `Tag` 在声明语境（`struct Tag x;`）中保持为 `Tag` 非终结符；
  - 在必要时显式创建中间非终结符节点（`Tag`、`OptTag`）以匹配期望输出。

### 3. 恢复策略的调试难点

- 为了能在报错后继续解析，需要词法器与语法器配合返回合理的替代 token（如非法十六进制返回 `INT_CONST(0)`），并在语法规则中通过 `error` 产生式剪枝恢复。

示例构建与运行命令：

```
cd ../Code
make
./parser <input.cmm>
```

## 六、总结与后续工作

本次实验的重点是保证语义动作精确地生成评测所需的 AST 文本形式，并提供稳定的错误恢复策略。实现过程中最花时间的部分不是 lexer 或 parser 本身的机械实现，而是不断微调语法动作的细节以满足自动化比对的严格格式要求。